

PATRONES DE DISEÑO

Patrón Creacional Factory Method

Es un patrón de diseño creacional que proporciona una interfaz para crear objetos en una superclase, mientras permite a las subclases alterar el tipo de objetos que se crearán.

El patrón Factory Method sugiere que, en lugar de llamar al operador new para construir objetos directamente, se invoque a un método fábrica especial. No te preocupes: los objetos se siguen creando a través del operador new, pero se invocan desde el método fábrica. Los objetos devueltos por el método fábrica a menudo se denominan productos.

Aplicabilidad:

Se utiliza cuando las dependencias y los tipos de los objetos con los que se va a trabajar se desconocen.

Cuando se quiera ofrecer a los usuarios de la biblioteca o framework, una forma de extender sus componentes internos.

Cuando se necesite ahorrar recursos del sistema mediante reutilización de objetos existentes en lugar de reconstruirlos cada vez.

Pros

1. Evitas un acoplamiento fuerte entre el creador y los productos concretos.
2. *Principio de responsabilidad única.* Puedes mover el código de creación de producto a un lugar del programa, haciendo que el código sea más fácil de mantener.
3. *Principio de abierto/cerrado.* Puedes incorporar nuevos tipos de productos en el programa sin descomponer el código cliente existente.

Contras

1. Puede ser que el código se complique, ya que debes incorporar una multitud de nuevas subclases para implementar el patrón. La situación ideal sería introducir el patrón en una jerarquía existente de clases creadoras.

Patrón Creacional Abstract Factory

Es un patrón de diseño creacional que nos permite producir familias de objetos relacionados sin especificar sus clases concretas.

Este método sugiere que declaremos de forma explícita interfaces para cada producto diferente de la familia de productos (por ejemplo, silla, sofá o mesilla). Después podemos hacer que todas las variantes de los productos sigan esas interfaces. Por ejemplo, todas las variantes de silla pueden implementar la interfaz Silla, así como todas las variantes de mesilla pueden implementar la interfaz Mesilla, y así sucesivamente.

Aplicabilidad:

Se utiliza cuando el código deba funcionar con varias familias de productos relacionados, pero no debe depender de las clases concretas de esos productos, ya que puede ser que se conozca de antemano.

Cuando se tenga una clase con un grupo de métodos de fábrica que nublen su responsabilidad principal.

Pros

1. Puedes tener la certeza de que los productos que obtienes de una fábrica son compatibles entre sí.
2. Evitas un acoplamiento fuerte entre productos concretos y el código cliente.
3. *Principio de responsabilidad única*. Puedes mover el código de creación de productos a un solo lugar, haciendo que el código sea más fácil de mantener.
4. *Principio de abierto/cerrado*. Puedes introducir nuevas variantes de productos sin descomponer el código cliente existente.

Contras

1. Puede ser que el código se complique más de lo que debería, ya que se introducen muchas nuevas interfaces y clases junto al patrón.

Patrón Creacional Builder

Nos permite construir objetos complejos paso a paso. El patrón nos permite producir distintos tipos y representaciones de un objeto empleando el mismo código de construcción. Este patrón nos sugiere que saquemos el código de construcción del objeto de su propia clase y lo coloquemos dentro de objetos independientes llamados constructores.

Aplicabilidad:

Se utiliza el patrón Builder para evitar un “constructor telescópico”.

Se utiliza cuando se necesite que el código sea capaz de crear distintas representaciones de ciertos productos.

Se utiliza para construir arboles con el patrón composite u otros objetos complejos.

Pros

1. Puedes construir objetos paso a paso, aplazar pasos de la construcción o ejecutar pasos de forma recursiva.
2. Puedes reutilizar el mismo código de construcción al construir varias representaciones de productos.
3. *Principio de responsabilidad única.* Puedes aislar un código de construcción complejo de la lógica de negocio del producto.

Contras

1. La complejidad general del código aumenta, ya que el patrón exige la creación de varias clases nuevas.

Patrón Creacional Prototype

Es un patrón de diseño creacional que nos permite copiar objetos existentes sin que el código dependa de sus clases.

Aplicabilidad

Se utiliza cuando el código no deba depender de las clases concretas de objetos que se necesiten copiar.

Se utiliza cuando sea útil reducir la cantidad de subclases que solo se diferencian en la forma en que se inicializan los proyectos.

Pros

1. Puedes clonar objetos sin acoplarlos a sus clases concretas.
2. Puedes evitar un código de inicialización repetido clonando prototipos prefabricados.
3. Puedes crear objetos complejos con más facilidad.
4. Obtienes una alternativa a la herencia al tratar con preajustes de configuración para objetos complejos.

Contras

1. Clonar objetos complejos con referencias circulares puede resultar complicado.

Patrón creacional Singleton:

También llamado instancia única nos permite asegurarnos de que cada clase tenga una única instancia, a la vez que proporciona un punto de acceso global a dicha instancia. Nos ayuda a la resolución de dos problemas al mismo tiempo, vulnerando el principio de la responsabilidad única:

1. Garantiza que una clase tenga una única instancia.
2. Proporciona un punto de acceso global a dicha instancia.

Pasos a considerar:

1. Hacer un constructor privado por defecto para evitar que otros objetos utilicen el operador new con la clase singleton.
2. Crear un método de creación estático que actúe como constructor, esto hace que el método invoque al constructor privado para crear un objeto y lo guarda en un campo estático.

Analogía del mundo real

Un país sólo puede tener un gobierno oficial. Independientemente de las identidades personales de los individuos que forman el gobierno, el título “Gobierno de X” es un punto de acceso global que identifica al grupo de personas a cargo.

Aplicabilidad

Cuando una clase del programa deba tener una instancia disponible para todos los clientes; por ejemplo, un único objeto de base de datos compartido por distintas partes del programa.

Cuando se requiera de un control muy estricto de las variables globales.

Pros

- Puedes tener la certeza de que una clase tiene una única instancia.
- Obtienes un punto de acceso global a dicha instancia.
- El objeto Singleton solo se inicializa cuando se requiere por primera vez.

Contras

1. Vulnera el Principio de responsabilidad única. El patrón resuelve dos problemas al mismo tiempo.
2. El patrón Singleton puede enmascarar un mal diseño, por ejemplo, cuando los componentes del programa saben demasiado los unos sobre los otros.
3. El patrón requiere de un tratamiento especial en un entorno con múltiples hilos de ejecución, para que varios hilos no creen un objeto Singleton varias veces.
4. Puede resultar complicado realizar la prueba unitaria del código del Singleton porque muchos frameworks de prueba dependen de la herencia a la hora de crear objetos simulados (mock objects). Debido a que la clase Singleton es privada y en la mayoría de los lenguajes resulta imposible sobrescribir métodos estáticos.

Patrón estructural Adapter

También llamado adaptador, Envoltorio, Wrapper tiene como propósito la colaboración entre objetos con interfaces incompatibles, es decir, envuelve uno de los objetos para esconder la complejidad de la conversión. El objeto envuelto ni siquiera es consciente de la existencia del adaptador. Por ejemplo, puedes envolver un objeto que opera con metros y kilómetros con un adaptador que convierte todos los datos al sistema anglosajón, es decir, pies y millas.

Funcionamiento:

1. El adaptador obtiene una interfaz compatible con uno de los objetos existentes.
2. Utilizando esta interfaz, el objeto existente puede invocar con seguridad los métodos del adaptador.
3. Al recibir una llamada, el adaptador pasa la solicitud al segundo objeto, pero en un formato y orden que ese segundo objeto espera.

Analogía del mundo real

Cuando viajas de Europa a Estados Unidos por primera vez, puede ser que te lleves una sorpresa cuanto intentes cargar tu computadora portátil. Los tipos de enchufe son diferentes en cada país, por lo que un enchufe español no sirve en Estados Unidos. El problema puede solucionarse utilizando un adaptador que incluya el enchufe americano y el europeo.

Aplicabilidad

Cuando quieras usar una clase existente, pero cuya interfaz no sea compatible con el resto del código.

Cuando se requiera reutilizar varias subclases existentes que carezcan de alguna funcionalidad común que no pueda añadirse a la superclase.

Pros

1. Principio de responsabilidad única. Se puede separar la interfaz o el código de conversión de datos de la lógica de negocio primaria del programa.
2. Principio de abierto/cerrado. Se puede introducir nuevos tipos de adaptadores al programa sin descomponer el código cliente existente, siempre y cuando trabajen con los adaptadores a través de la interfaz con el cliente.

Contras

1. La complejidad general del código aumenta, ya que se debe introducir un grupo de nuevas interfaces y clases. En ocasiones resulta más sencillo cambiar la clase de servicio de modo que coincida con el resto de tu código.

Patrón estructural Bridge

También conocido como puente, ya que permite dividir una clase grande, o un grupo de clases estrechamente relacionadas, en dos jerarquías separadas (abstracción e implementación) que pueden desarrollarse independientemente la una a la otra.

El patrón Bridge intenta resolver este problema pasando de la herencia a la composición del objeto. Esto quiere decir que se extrae una de las dimensiones a una jerarquía de clases separada, de modo que las clases originales referencian un objeto de la nueva jerarquía, en lugar de tener todo su estado y sus funcionalidades dentro de una clase.

Aplicabilidad

Cuando se quiera dividir y organizar una clase monolítica que tenga muchas variantes de una sola funcionalidad (por ejemplo, si la clase puede trabajar con diversos servidores de bases de datos).

Cuando se necesite extender una clase en varias dimensiones ortogonales (independientes).

Cuando se necesite poder cambiar implementaciones durante el tiempo de ejecución.

Pros

1. Se puede crear clases y aplicaciones independientes de plataforma.
2. El código cliente funciona con abstracciones de alto nivel. No está expuesto a los detalles de la plataforma.
3. Principio de abierto/cerrado. Se puede introducir nuevas abstracciones e implementaciones independientes entre sí.
4. Principio de responsabilidad única. Se puede centrar en la lógica de alto nivel en la abstracción y en detalles de la plataforma en la implementación.

Contras

1. Puede ser que el código se complique si aplicas el patrón a una clase muy cohesionada.

Patrón estructural Composite

También llamado objeto compuesto, object tree permite componer objetos en estructuras de árbol y trabajar con esas estructuras como si fueran objetos individuales.

Consiste en representar objetos simples y sus containers (o contenedores, también llamados colecciones en algunos lenguajes, o sea: grupos de objetos) en una clase abstracta de manera que puedan ser tratados uniformemente.

Analogía del mundo real

Los ejércitos de la mayoría de países se estructuran como jerarquías. Un ejército está formado por varias divisiones; una división es un grupo de brigadas y una brigada está formada por pelotones, que pueden dividirse en escuadrones. Por último, un escuadrón es un pequeño grupo de soldados reales. Las órdenes se dan en la parte superior de la jerarquía y se pasan hacia abajo por cada nivel hasta que todos los soldados saben lo que hay que hacer.

Aplicabilidad

Cuando se tenga que implementar una estructura de objetos con forma de árbol.

Cuando se requiera que el código cliente trate elementos simples y complejos de la misma forma.

Pros

1. Puedes trabajar con estructuras de árbol complejas con mayor comodidad: utiliza el polimorfismo y la recursión en tu favor.
2. Principio de abierto/cerrado. Puedes introducir nuevos tipos de elemento en la aplicación sin descomponer el código existente, que ahora funciona con el árbol de objetos.

Contras

1. Puede resultar difícil proporcionar una interfaz común para clases cuya funcionalidad difiere demasiado. En algunos casos, tendrás que generalizar en exceso la interfaz componente, provocando que sea más difícil de comprender.

[Patrones de diseño / Design patterns](#)

[Patrón Composite: definición, diagrama UML y ejemplos - IONOS España](#)